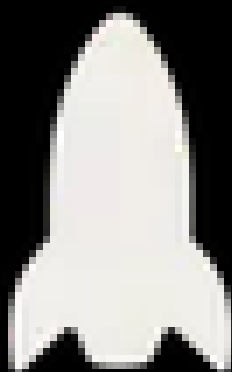




KICKOFF

Security Review



JANUARY, 2026

www.chaindefenders.xyz
<https://x.com/DefendersAudits>

Lead Auditors



PeterSR



0x539.eth

Table of Contents

1. Protocol Summary
2. Disclaimer
3. Risk Classification
4. Audit Details
 - Commit Hash
 - Scope
 - Roles
5. Executive Summary
 - Issues Found
 - Issues Statuses
 - Security Grade
6. Findings
 - Low
 - Informational
 - Gas

Protocol Summary

A liquidity bootstrapping launchpad that leverages Aerodrome's veAERO governance on Base.

Kickoff enables projects to bootstrap liquidity by leveraging veAERO voting power:

- Projects create tokens with customizable tokenomics via Token Factory
- Projects deposit tokens and create a Vote-Sale Pool
- veAERO holders lock their NFTs to provide voting power

- Voting power is used to vote for the project's gauge on Aerodrome
- Rewards (bribes + fees) are converted to WETH and paired with project tokens
- LP tokens are permanently locked, generating trading fees forever
- Participants claim project tokens proportional to their voting power contribution

Disclaimer

The Chain Defenders team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the implementation of the contracts.

Risk Classification

Impact/Likelihood	High	Medium	Low
High	High	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

Audit Details

Commit Hash

1655ff70ea5453fc6658ad66691cfd87936dcf54

Scope

Id	Files in scope
1	./script/Deploy.s.sol
2	./src/ProjectToken.sol
3	./src/ProjectTokenFactory.sol
4	./src/TokenVesting.sol

Roles

Id	Roles
1	Owner
2	ProjectTokenFactory
3	User

Executive Summary

Issues Found

Severity	Count	Description
High	0	Critical vulnerabilities
Medium	0	Significant risks
Low	6	Minor issues with low impact
Informational	6	Best practices or suggestions
Gas	1	Optimization opportunities

Issues Statuses

Id	Title	Status
L-01	Plain Text Private Key In Deployment Script	Acknowledged
L-02	Wrong Revert When 'tgePercent' Is Zero And Recipient Is Zero Address	Acknowledged
L-03	Operational Friction Caused By Strict Allocation Sum Equality	Acknowledged
L-04	Missing Per Address Token Creation Limit Can Lead To Possible Spamming	Acknowledged
L-05	Missing Duplicate Name And Symbol Check When Creating Tokens	Acknowledged
L-06	Owner Has Exclusive Control Over Starting Token Vesting	Acknowledged
I-01	Ability To Create Vesting Locks After TGE Start	Acknowledged
I-02	Events Are Not Emitted On State Changes	Acknowledged
I-03	Missing Checks For Empty '_name' Or '_symbol'	Acknowledged
I-04	'TokenAlreadyExists' Is Not Used	Acknowledged
I-05	Wrong Error Name For Caller Validation Against Pending Owner	Acknowledged
I-06	Wrong Error Name For Insufficient Rescuable Amount	Acknowledged
G-01	Deployer Balance Not Being Validated Could Lead To Wasted Gas	Acknowledged

Security Grade



Based on the audit findings and code quality, the overall protocol security grade is **94.00 / 100.00**. This reflects an excellent security posture with almost no room for improvement.

Findings

Low

Low 01 Plain Text Private Key In Deployment Script

Location

Deploy.s.sol

Description

The deployment script directly retrieves the deployer's private key from environment variables using `vm.envUint("PRIVATE_KEY")`. This practice exposes sensitive information to multiple security risks:

1. **Shell History:** Private keys stored in environment variables can be logged in shell command history, making them accessible through `.bash_history`, `.zsh_history`, or similar files.
2. **Process Exposure:** Environment variables are visible in process listings and system monitoring tools, potentially exposing the key to other users on shared systems.
3. **.env File Leaks:** When using `.env` files to populate environment variables, these files often contain plain-text private keys and can be accidentally committed to version control or exposed through misconfigured backups.
4. **Log Files and Error Messages:** Environment variables can leak into application logs, error traces, or CI/CD pipeline outputs if exceptions occur during script execution.
5. **Memory Exposure:** The private key remains in memory during script execution and could be captured through memory dumps or debugging tools.

This is not a functional vulnerability in the contract logic itself, but rather an operational security issue in the deployment process that could lead to complete loss of funds and control over deployed contracts.

Recommendation

Implement more secure private key management:

1. **Foundry Keystore (Recommended for Local Development):** Use Foundry's built-in keystore with encrypted storage.
2. **Interactive Prompt:** Have the deployer enter the key at runtime rather than from environment variables, keeping it out of shell history and process listings.
3. **Hardware Wallets/Ledger Integration:** Use hardware wallet integrations with Foundry for production deployments.
4. **Secrets Management Service:** For automated deployments, use AWS Secrets Manager, HashiCorp Vault, or similar services with proper access controls and audit logging.

Status

Acknowledged

Low 02 Wrong Revert When tgePercent Is Zero And Recipient Is Zero Address

Location

ProjectTokenFactory.sol:160

Description

The `createToken` function enforces a blanket validation that `alloc.recipient` $\neq$ `address(0)`, rejecting any allocation with a zero address as the TGE recipient. However, this validation is overly strict when `tgePercent` is 0 (zero percent unlocked at TGE).

When `tgePercent` = 0:

- `tgeAmount` = `(alloc.amount * 0) / BPS_DENOMINATOR` = 0
- In `_distributeAllocation`, the condition `if (tgeAmount > 0)` is never true
- No tokens are ever transferred to `alloc.recipient`
- The entire allocation amount becomes vesting tokens for `alloc.lockOwner`

In this scenario, the `recipient` field is completely unused. The requirement to provide a non-zero recipient address adds unnecessary friction and is semantically incorrect - if no tokens are being distributed at TGE, the recipient address is irrelevant.

Impacts:

1. Operational Friction: Users must provide a dummy recipient address even when creating pure vesting allocations (0% TGE), making the API less intuitive.
2. Logic Inconsistency: The validation doesn't match the actual behavior - tokens aren't sent to recipient if `tgePercent` is 0, so why require it?
3. Code Clarity: The validation obscures the intent that vesting-only allocations have no immediate recipient.

Recommendation

Conditionally allow `address(0)` as a recipient when `tgePercent` = 0:

```
1 // Validate allocations sum to totalSupply
2 uint256 allocationSum = 0;
3 for (uint256 i = 0; i < allocations.length;) {
4     TokenAllocation calldata alloc = allocations[i];
5 }
```



```
6     if (alloc.amount == 0) revert ZeroAmount();
7     // Only require recipient if tokens are unlocked at TGE
8     if (alloc.tgePercent > 0 && alloc.recipient == address(0)) revert ZeroAddress();
9     if (alloc.tgePercent > BPS_DENOMINATOR) revert InvalidAllocation();
10
11    // If there's vesting (not 100% TGE), lockOwner must be set
12    if (alloc.tgePercent < BPS_DENOMINATOR && alloc.lockOwner == address(0)) {
13        revert ZeroAddress();
14    }
15
16    allocationSum += alloc.amount;
17    unchecked { ++i; }
18 }
```

This allows creating pure vesting allocations (0% TGE, 100% vesting) without requiring a placeholder recipient address, making the API cleaner and more logically consistent with the actual token distribution behavior.

Status

Acknowledged

Low 03 Operational Friction Caused By Strict Allocation Sum Equality

Location

ProjectTokenFactory.sol:172

Description

The `createToken` function enforces strict equality, requiring that the sum of all allocation amounts match the total supply exactly:

```
1 if (allocationSum != totalSupply) revert AllocationMismatch();
```

This creates a critical UX friction point when allocating tokens with percentages. Consider a real-world scenario:

- Total supply: 1,000,000 tokens
- Allocation 1: 50% to address A = 500,000 tokens

- Allocation 2: 33.33% to address B = 333,300 tokens
- Allocation 3: 16.67% to address C = 166,700 tokens
- Total: 1,000,000 (but integer division in actual code could yield 999,999)

Due to integer rounding in Solidity, percentage-based allocations frequently result in “dust” - leftover tokens that don’t get allocated. A user with a properly designed tokenomics spreadsheet can legitimately end up with `allocationSum = 999,999` and `totalSupply = 1,000,000`, causing the transaction to fail entirely.

Impacts:

1. Transaction Failure: Valid tokenomics configurations are rejected due to rounding errors beyond the user’s control.
2. Poor UX: Users must manually adjust percentages to force exact matches, complicating token deployment.
3. Operational Friction: Users may need to redeploy tokens multiple times with different numbers to find an exact match.

Recommendation

Modify the validation to allow `allocationSum ≤ totalSupply` and distribute any remainder to the creator:

```
1 // Allow allocations ≤ totalSupply; remainder goes to creator
2 if (allocationSum > totalSupply) revert AllocationMismatch();
3
4 // Send any dust/remainder to creator
5 uint256 dust = totalSupply - allocationSum;
6 if (dust > 0) {
7     bool success = IERC20(token).transfer(msg.sender, dust);
8     if (!success) revert TransferFailed();
9 }
```

This approach:

- Accepts any valid allocation spreadsheet without forcing exact sums
- Automatically handles rounding dust by sending it to the creator
- Follows industry standard patterns used by major token factories
- Improves UX and reduces deployment failures

Status

Acknowledged

Low 04 Missing Per Address Token Creation Limit Can Lead To Possible Spamming

Location

ProjectTokenFactory.sol

Description

The `createToken` function has no restrictions on how many tokens a single user can create. Each successful deployment pushes a new token address to the `allTokens` array:

```
1 allTokens.push(token);
2 isToken[token] = true;
3 creatorTokens[msg.sender].push(token);
```

There is no per-creator limit, no global cap, and no access control. This creates two problems:

1. Unbounded Array Growth

- The `allTokens` array grows indefinitely with each token creation
- Any user can spam the factory by creating thousands of tokens
- After 10,000+ deployments, the array becomes massive in storage

2. Gas-Intensive View Functions The `getAllTokens` function returns the entire array in memory:

```
1 function getAllTokens() external view returns (address[] memory) {
2     return allTokens;
3 }
```

- Calling this function on a factory with 100,000 tokens becomes prohibitively expensive
- Block explorers and frontends that read this data will timeout or fail
- Users querying token history face escalating gas costs

Real-World Impact:

- Malicious actor creates 50,000 spam tokens -> `getAllTokens` becomes unusable
- Legitimate users cannot query the factory's token list
- The contract becomes effectively broken for its primary use case
- Other contracts integrating with this factory experience failures

Recommendation

Implement a per-creator token limit and optionally a global cap:

```
1  // Add constants
2  uint256 public constant MAX_TOKENS_PER_CREATOR = 100;
3  uint256 public constant MAX_TOKENS_GLOBAL = 10000;
4
5  // Add mapping to track creator's token count
6  mapping(address => uint256) public tokenCountByCreator;
7
8  // In createToken(), add these checks before deployment:
9  function createToken(
10     string calldata name,
11     string calldata symbol,
12     uint256 totalSupply,
13     TokenAllocation[] calldata allocations
14 ) external returns (address token) {
15     if (totalSupply == 0) revert ZeroAmount();
16     if (allocations.length == 0) revert InvalidAllocation();
17     if (allocations.length > MAX_ALLOCATIONS) revert TooManyAllocations();
18
19     // Add new limit checks
20     if (tokenCountByCreator[msg.sender] ≥ MAX_TOKENS_PER_CREATOR) {
21         revert TooManyTokens(); // New error
22     }
23     if (allTokens.length ≥ MAX_TOKENS_GLOBAL) {
24         revert GlobalTokenLimitReached(); // New error
25     }
26
27     // ... rest of validation ...
28
29     // Deploy token
30     token = address(new ProjectToken(name, symbol, totalSupply, address(this)));
31
32     // Register token
33     allTokens.push(token);
34     isToken[token] = true;
35     creatorTokens[msg.sender].push(token);
36     tokenCountByCreator[msg.sender]++; // Track creator's count
37
38     // ... rest of function ...
39 }
```

Status

Acknowledged

Low 05 Missing Duplicate Name And Symbol Check When Creating Tokens

Location

ProjectTokenFactory.sol

Description

The `createToken` function accepts any token name and symbol from callers without validating against duplicates. A malicious user can create tokens that perfectly impersonate existing, legitimate tokens:

Example Attack:

- User A creates: `name = "Token A", symbol = "TA"`
- User B creates: `name = "Token A", symbol = "TA"` (mimicking User A's token)

The deployment succeeds without warnings or blocking. The contract simply records the information:

```
1 tokenInfo[token] = TokenInfo({
2     token: token,
3     name: name,
4     symbol: symbol,
5     totalSupply: totalSupply,
6     creator: msg.sender,
7     createdAt: block.timestamp
8 });
```

Impact:

- Victims lose funds buying or trading the wrong token
- Legitimate token projects suffer reputational damage
- Users can't trust the factory's token registry

Recommendation

Implement duplicate detection by tracking unique (name, symbol) combinations:

```
1 // Add error
2 error DuplicateToken();
3
4 // Add mapping to track existing name/symbol combos
```

```
5 mapping(bytes32 => bool) public tokenNameSymbolExists;
6
7 // In createToken(), add validation:
8 function createToken(
9     string calldata name,
10    string calldata symbol,
11    uint256 totalSupply,
12    TokenAllocation[] calldata allocations
13 ) external returns (address token) {
14     if (totalSupply == 0) revert ZeroAmount();
15     if (allocations.length == 0) revert InvalidAllocation();
16     if (allocations.length > MAX_ALLOCATIONS) revert TooManyAllocations();
17
18     // Check for duplicate name/symbol combination
19     bytes32 nameSymbolHash = keccak256(abi.encodePacked(name, symbol));
20     if (tokenNameSymbolExists[nameSymbolHash]) revert DuplicateToken();
21
22     // ... rest of validation ...
23
24     // Deploy token
25     token = address(new ProjectToken(name, symbol, totalSupply, address(this)));
26
27     // Register token and mark name/symbol as used
28     allTokens.push(token);
29     isToken[token] = true;
30     creatorTokens[msg.sender].push(token);
31     tokenNameSymbolExists[nameSymbolHash] = true; // Prevent future duplicates
32
33     tokenInfo[token] = TokenInfo({
34         token: token,
35         name: name,
36         symbol: symbol,
37         totalSupply: totalSupply,
38         creator: msg.sender,
39         createdAt: block.timestamp
40     });
41
42     emit TokenCreated(token, name, symbol, totalSupply, msg.sender);
43
44     // Distribute allocations ...
45 }
```

Status

Acknowledged

Low 06 Owner Has Exclusive Control Over Starting Token Vesting

Location

TokenVesting.sol

Description

The `startVesting` function is the critical gatekeeper for token vesting, yet it's exclusively controlled by the owner with no fallback mechanism:

```
1 function startVesting(address token) external onlyOwner {
2     if (token == address(0)) revert ZeroAddress();
3     if (tgeStart[token] != 0) revert VestingAlreadyStarted();
4
5     tgeStart[token] = block.timestamp;
6     emit VestingStarted(token, block.timestamp);
7 }
```

If the owner never calls this function, `tgeStart[token]` remains 0. When beneficiaries attempt to claim vested tokens, the `_getClaimable()` function returns 0:

```
1 function _getClaimable(VestingSchedule storage schedule) internal view returns (uint256)
2     {
3         uint256 start = tgeStart[schedule.token];
4
5         // Vesting not started
6         if (start == 0) return 0; // <- Always returns 0 if never started
7
8         // ... rest of vesting logic ...
9     }
```

Attack/Risk Scenarios:

1. Negligence: Owner forgets or is unaware they must call `startVesting` -> tokens remain locked permanently
2. Key Loss: Owner's private keys are lost/compromised -> vesting can never start
3. Malicious Delay: Owner deliberately delays calling `startVesting` to:
 - Extract strategic value from token holders who can't access their vested tokens
 - Create uncertainty/FUD around the project
 - Perform a "soft rug" by locking legitimate allocations indefinitely

4. Multi-token Issue: If tokens are created over time, the owner must remember to call `startVesting` for each one

Recommendation

Implement a decentralized vesting start mechanism:

```
1 // Vesting starts automatically at deployment time + fixed delay
2 uint256 public constant MAX_VESTING_DELAY = 30 days;
3
4 function createToken( ... ) external returns (address token) {
5     // ... token creation logic ...
6
7     // Automatically set vesting to start 30 days after deployment
8     tokenPresetStartTime[token] = block.timestamp + MAX_VESTING_DELAY;
9 }
```

Status

Acknowledged

Informational

Info 01 Ability To Create Vesting Locks After TGE Start

Location

TokenVesting.sol

Description

The `createLock` function (restricted to the factory) can be called after vesting has started for a token (`tgeStart[token]  $\neq$  0`).

New locks created post-start will have their cliff and vesting periods partially or fully elapsed from the original start timestamp, allowing beneficiaries to immediately claim more (or even all) tokens than intended.

This breaks expected vesting schedules for late allocations and can lead to unintended early unlocks or tokenomics inconsistencies.

Even though this is not currently possible, it could become an issue if the invocation method of this function changes in the future.

Recommendation

Prevent new locks after vesting has started by adding a check in `createLock`:

```
1 if (tgeStart[token] != 0) {  
2     revert VestingAlreadyStartedForToken();  
3 }
```

Status

Acknowledged

Info 02 Events Are Not Emitted On State Changes

Location

TokenVesting.sol

Description

Several important state-changing operations in the `TokenVesting` contract do not emit events.

Specifically, the following critical actions lack dedicated events:

- Final ownership transfer completion (`acceptOwnership`): No `OwnershipTransferred` event is emitted when ownership actually changes hands.
- Vesting lock completion: No event is emitted when a lock reaches `claimed = totalAmount` (full vesting claimed).
- Ownership transfer initiation (`transferOwnership`): No event signals that a pending owner has been proposed (only the internal state changes).

Current code does not emit events in:

- `acceptOwnership`
- `_claim` (when `schedule.claimed = schedule.totalAmount` after update)
- `transferOwnership`

Recommendation

Add the following events and emissions for better transparency and standard compliance:

1. Define new events (add to contract events section):

```
1 event OwnershipTransferred(address indexed previousOwner, address indexed newOwner);
2 event OwnershipTransferStarted(address indexed previousOwner, address indexed
    pendingOwner);
3 event VestingCompleted(uint256 indexed lockId, address indexed token, address indexed
    beneficiary);
```

2. Add these events to the functions.

Status

Acknowledged

Info 03 Missing Checks For Empty `_name` Or `_symbol`

Location

ProjectToken.sol

ProjectTokenFactory.sol

Description

The `ProjectToken` constructor accepts `_name` and `_symbol` parameters but does not validate that these strings are non-empty before assignment. This is not validated in the `ProjectTokenFactory` contract also. This allows a token to be created with an empty name and/or symbol:

```
1 constructor(string memory _name, string memory _symbol, uint256 _totalSupply, address
    _mintTo) {
2     if (_mintTo == address(0)) revert ZeroAddress();
3
4     // No validation of _name or _symbol length
5     name = _name;
6     symbol = _symbol;
7     totalSupply = _totalSupply;
8     balanceOf[_mintTo] = _totalSupply;
9 }
```

Impacts:

1. UX Issues: Block explorers, wallets, and DEX interfaces may display the token incorrectly or show empty fields, confusing users about what token they're interacting with.
2. Data Integrity: A token with an empty name/symbol violates the semantic intention of these fields and represents incomplete or malformed token metadata.

3. User Confusion: Multiple tokens with empty names could be confused with each other in wallet displays.

Recommendation

Add validation in the constructor to ensure `_name` and `_symbol` are non-empty:

```
1 error EmptyName();
2 error EmptySymbol();
3
4 constructor(string memory _name, string memory _symbol, uint256 _totalSupply, address
   _mintTo) {
5     if (_mintTo == address(0)) revert ZeroAddress();
6     if (bytes(_name).length == 0) revert EmptyName();
7     if (bytes(_symbol).length == 0) revert EmptySymbol();
8
9     name = _name;
10    symbol = _symbol;
11    totalSupply = _totalSupply;
12    balanceOf[_mintTo] = _totalSupply;
13
14    emit Transfer(address(0), _mintTo, _totalSupply);
15 }
```

This ensures every deployed token has valid, non-empty metadata from inception, preventing confusion and maintaining data quality across the protocol.

Status

Acknowledged

Info 04 TokenAlreadyExists Is Not Used

Location

ProjectTokenFactory.sol

Description

The `TokenAlreadyExists` error is declared but is never actually used anywhere in the contract:

```
1 error TokenAlreadyExists();
```

A search through the entire `ProjectTokenFactory` contract reveals no instances where this error is thrown or checked. The `createToken()` function has no logic to prevent duplicate token names or symbols, nor does it use this error to validate any conditions.

Recommendation

Remove the unused error definition:

```
1 error ZeroAddress();
2 error ZeroAmount();
3 error NotOwner();
4 error InvalidAllocation();
5 error AllocationMismatch();
6 error TransferFailed();
7 // Remove this line:
8 // error TokenAlreadyExists();
9 error TooManyAllocations();
```

Alternative: If preventing duplicate token names/symbols was a planned feature, implement the full validation logic with this error.

Status

Acknowledged

Info 05 Wrong Error Name For Caller Validation Against Pending Owner

Location

`ProjectTokenFactory.sol`

Description

The `acceptOwnership` function uses a misleading error name:

```
1 function acceptOwnership() external {
2     // Misleading error name
3     if (msg.sender != pendingOwner) revert NotOwner();
4
5     address oldOwner = owner;
6     owner = msg.sender;
7     pendingOwner = address(0);
8
9     emit OwnershipTransferred(oldOwner, msg.sender);
10 }
```

The validation checks if the caller is the pending owner, but throws the `NotOwner` error. This creates semantic confusion:

The Problem:

1. `NotOwner` suggests “you are not the current owner” (the person in the `owner` state variable)
2. But the actual check is “you are not the pending owner” (a different role with different permissions)
3. Someone reading the error message would be confused about why they can’t accept ownership when they have no claim to do so

Recommendation

Add a dedicated error for clarity and use it consistently:

```
1 // In ERRORS section
2 error NotOwner();
3 error NotPendingOwner(); // Add this new error
4
5 // In acceptOwnership()
6 function acceptOwnership() external {
7     if (msg.sender != pendingOwner) revert NotPendingOwner(); // Use specific error
8
9     address oldOwner = owner;
10    owner = msg.sender;
11    pendingOwner = address(0);
12
13    emit OwnershipTransferred(oldOwner, msg.sender);
14 }
```

Status

Acknowledged

Info 06 Wrong Error Name For Insufficient Rescuable Amount

Location

TokenVesting.sol

Description

The `emergencyRescue` function uses a semantically misleading error when validating the rescue amount:

```
1 function emergencyRescue(address token, address to, uint256 amount) external onlyOwner {
2     if (to == address(0)) revert ZeroAddress();
3
4     uint256 actuallyLocked = totalLocked[token] - totalClaimed[token];
5     uint256 balance = IERC20(token).balanceOf(address(this));
6     uint256 rescuable = balance > actuallyLocked ? balance - actuallyLocked : 0;
7
8     if (amount > rescuable) revert ZeroAmount(); // <- Semantically incorrect
9
10    bool success = IERC20(token).transfer(to, amount);
11    if (!success) revert TransferFailed();
12 }
```

The condition checks if `amount > rescuable`, but throws `ZeroAmount()`. This is semantically incorrect because:

The Problem:

1. `ZeroAmount()` implies “the amount parameter is zero” or “no amount was provided”
2. But the actual validation failure is “requested amount exceeds available rescuable tokens”
3. These are two entirely different error conditions

Recommendation

Add a dedicated error and use it for the correct validation:

```
1 // Add to error definitions
2 error InsufficientRescuableAmount();
3
4 // In emergencyRescue()
5 function emergencyRescue(address token, address to, uint256 amount) external onlyOwner {
6     if (to == address(0)) revert ZeroAddress();
7     if (amount == 0) revert ZeroAmount(); // Explicit check for zero amount
8 }
```

```
9      // Calculate how much is actually locked for this token
10     uint256 actuallyLocked = totalLocked[token] - totalClaimed[token];
11     uint256 balance = IERC20(token).balanceOf(address(this));
12
13     // Can only rescue excess tokens (tokens sent by mistake)
14     uint256 rescuable = balance > actuallyLocked ? balance - actuallyLocked : 0;
15     if (amount > rescuable) revert InsufficientRescuableAmount(); // Specific error
16
17     bool success = IERC20(token).transfer(to, amount);
18     if (!success) revert TransferFailed();
19 }
```

This follows the principle that error names should accurately reflect the condition that triggered them, improving code clarity and error handling reliability.

Status

Acknowledged

Gas

Gas 01 Deployer Balance Not Being Validated Could Lead To Wasted Gas

Location

Deploy.s.sol

Description

The deployment script displays the deployer's balance but performs no validation to ensure sufficient ETH exists for the deployment:

```
1 console.log("Balance:", deployer.balance / 1e15, "finney");
2 console.log("");
3
4 if (block.chainid == 84532) {
5     console.log("WARNING: Deploying to testnet with mainnet Aerodrome addresses.");
6     // ... warnings ...
7 }
8
```

```
9  vm.startBroadcast(deployerPrivateKey);
10 // Proceed directly to deployment without checking balance
```

The Problem:

1. No Pre-Flight Check: The script logs the balance for informational purposes only but continues unconditionally to deployment
2. Wasted Gas on Failure: If balance is insufficient, the script attempts deployment anyway, failing partway through and wasting gas

Recommendation

Add a balance validation check with estimated gas requirements:

```
1  // Add constant for estimated gas costs
2  uint256 private constant ESTIMATED_GAS_COST = 0.3 ether; // Adjust based on network
3
4  function run() external {
5      uint256 deployerPrivateKey = vm.envUint("PRIVATE_KEY");
6      address deployer = vm.addr(deployerPrivateKey);
7
8      // Determine network
9      string memory network = block.chainid == 8453 ? "Base Mainnet" :
10                             block.chainid == 84532 ? "Base Sepolia" : "Unknown";
11
12     console.log("");
13     console.log("=====");
14     console.log("  KICKOFF PROTOCOL - FULL DEPLOYMENT");
15     console.log("=====");
16     console.log("");
17     console.log("Network:", network);
18     console.log("Chain ID:", block.chainid);
19     console.log("Deployer:", deployer);
20     console.log("Balance:", deployer.balance / 1e15, "finney");
21
22     // Validate sufficient balance
23     uint256 estimatedGasCost = block.chainid == 8453 ? 0.3 ether : 0.1 ether; // Base
24     Mainnet vs Sepolia
25     if (deployer.balance < estimatedGasCost) {
26         console.log("");
27         console.log("ERROR: Insufficient balance for deployment!");
28         console.log("Required:", estimatedGasCost / 1e15, "finney");
29         console.log("Available:", deployer.balance / 1e15, "finney");
30         console.log("Shortfall:", (estimatedGasCost - deployer.balance) / 1e15, "finney");
31     };
32     revert("InsufficientBalance");
```



```
31     }
32
33     console.log("Balance sufficient for deployment");
34     console.log("");
35
36     if (block.chainid == 84532) {
37         console.log("WARNING: Deploying to testnet with mainnet Aerodrome addresses.");
38         console.log("        VoteSale contracts will deploy but won't be functional.");
39         console.log("        TokenFactory & Vesting will work normally.");
40         console.log("");
41     }
42
43     vm.startBroadcast(deployerPrivateKey);
44     // ... deployment logic ...
45 }
```

This is especially critical in production deployments where gas costs can be significant and partial deployments create protocol inconsistencies.

Status

Acknowledged